

LAB # 7

To Explain and Reproduce the Working of Switches and LED's on STM32F407 Trainer Board using STM32CubeIDE and HAL Library.

Objectives

- To explain the working of switch and LED interfacing with the STM32F407 using trainer board with STM32CubeIDE.
- To configure GPIO pins using STM32CubeMX integrated in STM32CubeIDE.
- To modify the program using HAL library functions and reproduce its output on STM32F407 trainer board.

Pre-Lab Exercise

Read the details given below in order to comprehend the basic operation of an LED, push button, and STM32CubeIDE development environment. Interfacing LED and push button with microcontroller can be performed using HAL library functions to accomplish different tasks in different combinations.

Introduction

Light Emitting Diodes (LEDs): Light Emitting Diodes are the most commonly used components in many applications. They are made of semiconducting material. A light-emitting diode (LED) is a two-lead semiconductor light source. It is a pn-junction diode, which emits light when activated. When a suitable voltage is applied to the leads, electrons are able to recombine with electron holes within the device, releasing energy in the form of photons. This effect is called electroluminescence, and the color of the light (corresponding to the energy of the photon) is determined by the energy band gap of the semiconductor.

- 1.9 to 2.1 V for red, orange and yellow,
- 3.0 to 3.4 V for green and blue,
- 2.9 to 4.2 V for violet, pink, purple and white.

We interface LED with microcontroller in two configurations (Source and Sink) as shown in Figure 7-1.

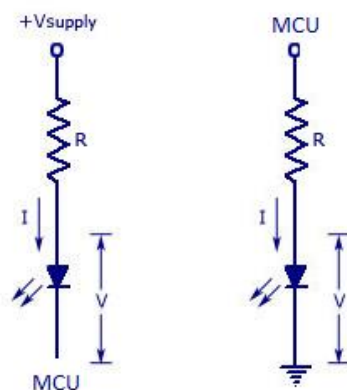


Figure 7-1: LED interfacing with microcontroller

Push Button Switches: The push-button switch is assumed to be bounce free, implying that when it is pressed and then released, a single pulse is produced. Push Button can interface in two types of configurations i.e. Pull up and Pull Down as shown in Figure 7-2. We use the terms pull-up or pull-down resistor, this is a simple resistor usually between 10k and 100k, to define the input state when no signal source is connected, this way overriding the noise signal, common sense dictates that when you have potentially larger noise then a smaller resistor is needed, but don't be careless about it, don't place a 100Ω resistor because your signal source must be able to "defeat" the pull-up (down) resistor. The rule of thumb is to use at least 10x larger pull-up (down) resistor then your signal source impedance.

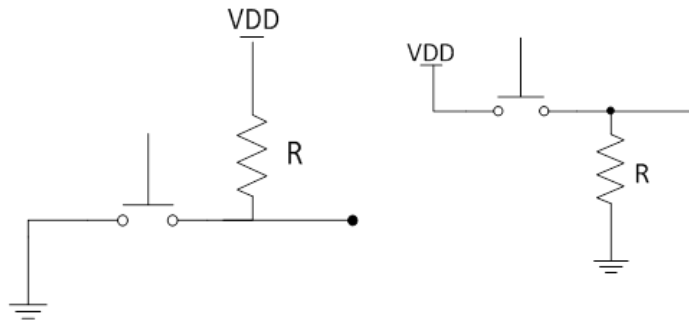


Figure 7-2: Push Button Configurations

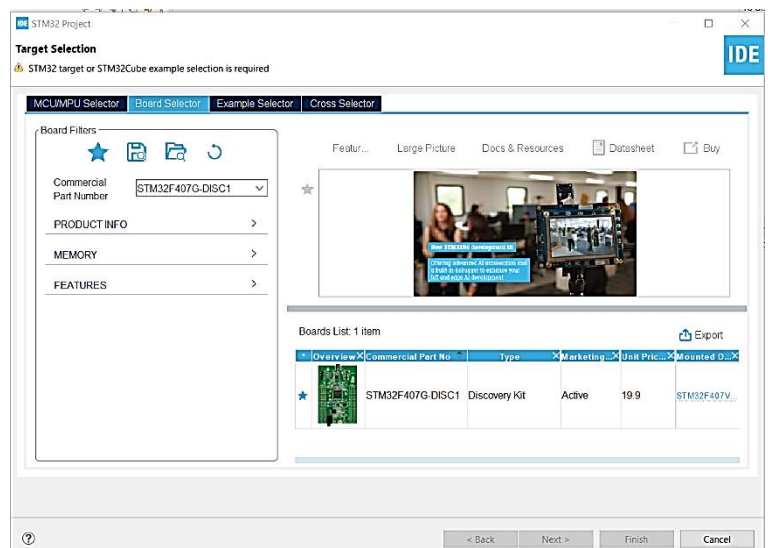
STM32CubeIDE Overview

STM32CubeIDE is an all-in-one multi-OS development tool, which is part of the STM32Cube software ecosystem. It integrates STM32CubeMX for graphical configuration of STM32 microcontrollers and includes a C/C++ development environment based on Eclipse/CDT with build analyzer, code editor, compiler, debugger, and flash programmer.

Creating a New Project in STM32CubeIDE

To create a new project:

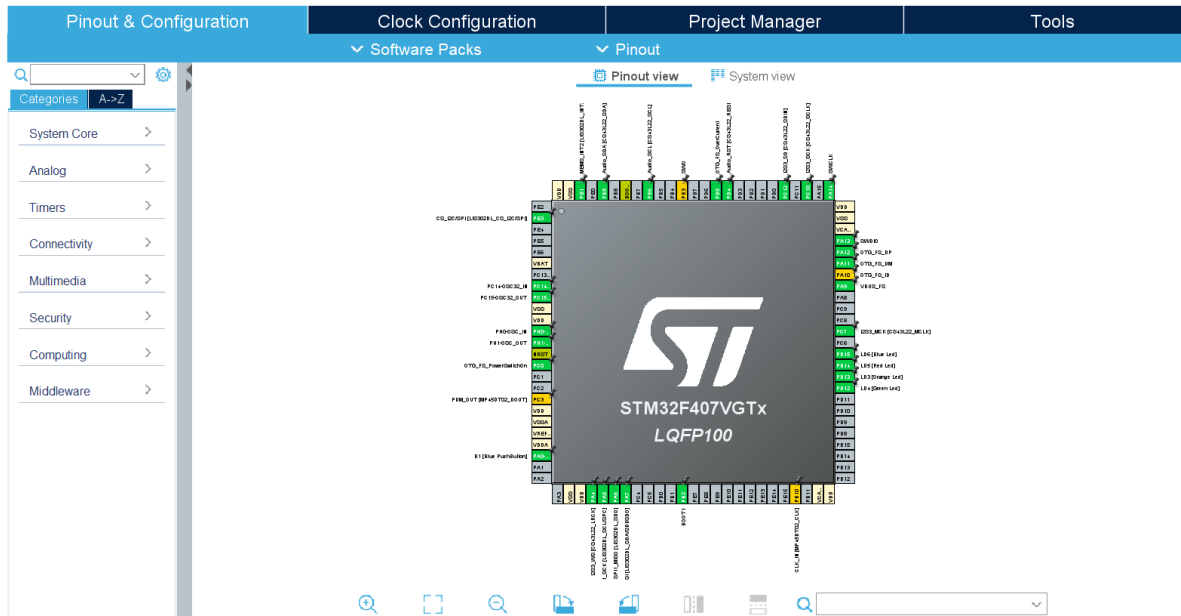
- Launch STM32CubeIDE
- Click File → New → STM32 Project
- Select your target STM32F407 microcontroller or board
- Enter project name and click Finish
- Select "Yes" when prompted to initialize all peripherals with default mode



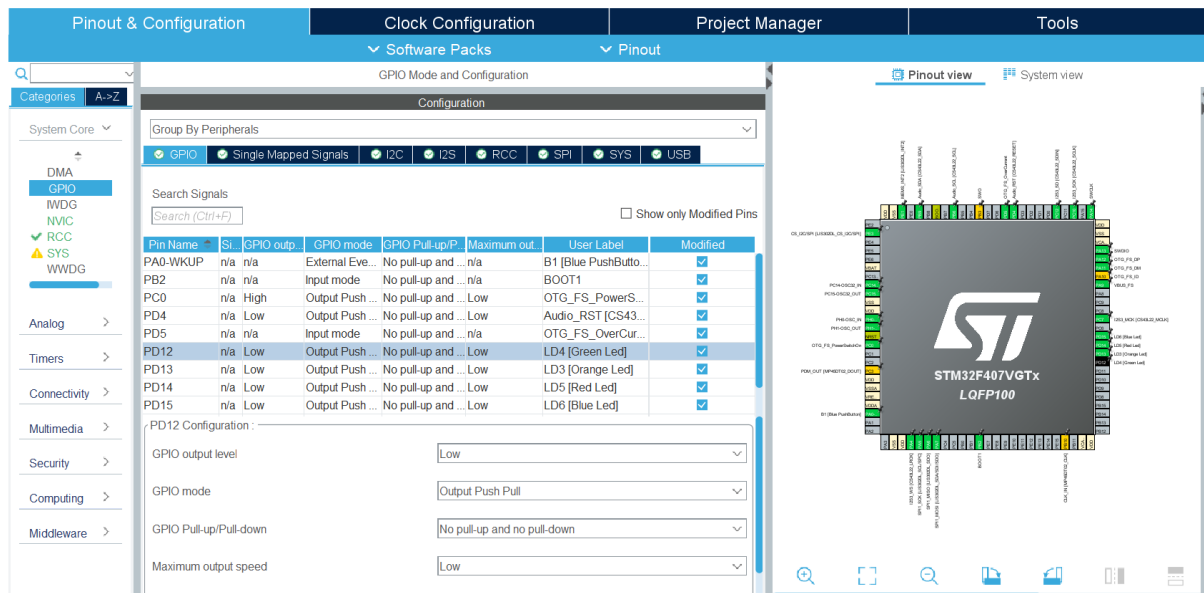
GPIO Configuration Using STM32CubeMX

STM32CubeMX provides a graphical interface to configure GPIO pins:

1. In the Pinout & Configuration tab, locate the desired pin on the chip diagram. PA0 for push button and PD12 for LED.

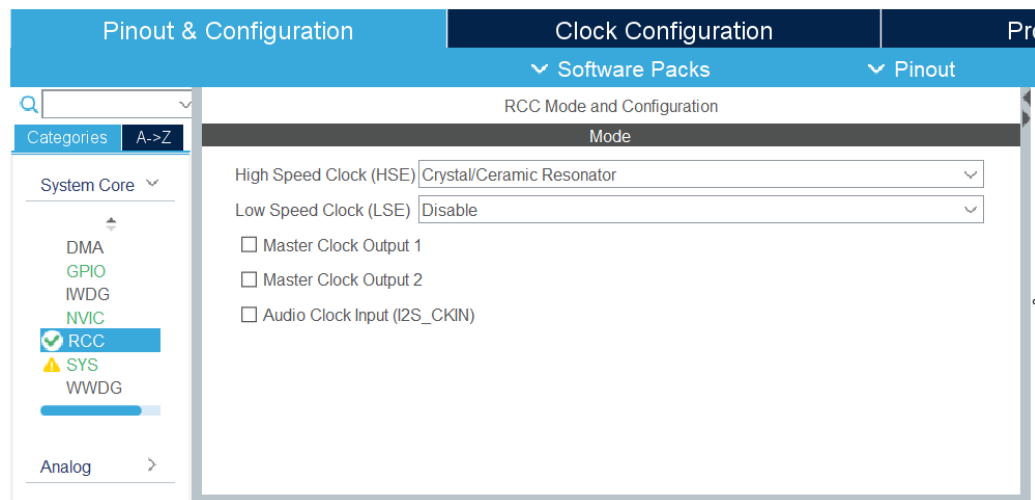


2. Click on the pin and select the desired mode (GPIO_Output for LED, GPIO_Input for button)

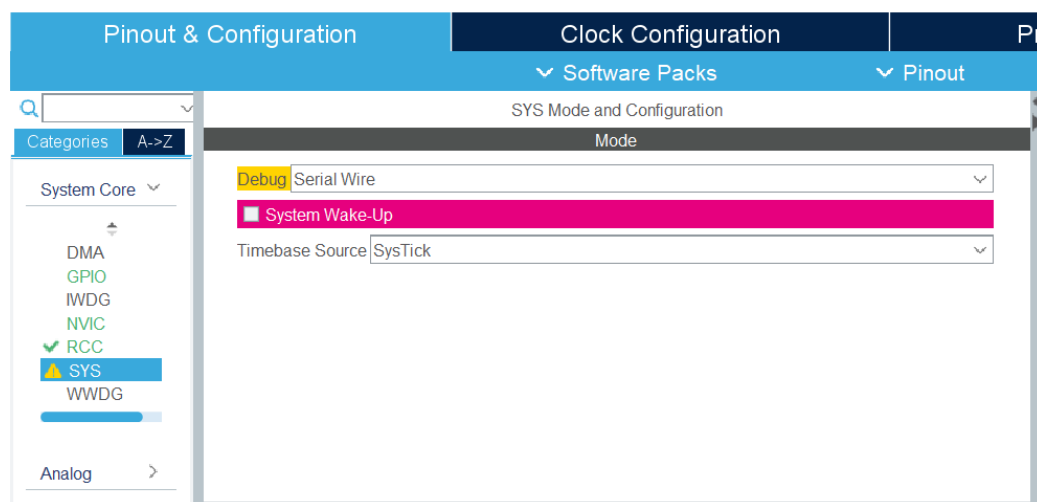


3. Configure GPIO parameters in the Configuration panel:
 - o GPIO Mode: Input/Output
 - o GPIO Pull-up/Pull-down: No pull-up/pull-down, Pull-up, or Pull-down
 - o GPIO Output Level: Low or High
 - o Maximum output speed: Low, Medium, High, or Very High

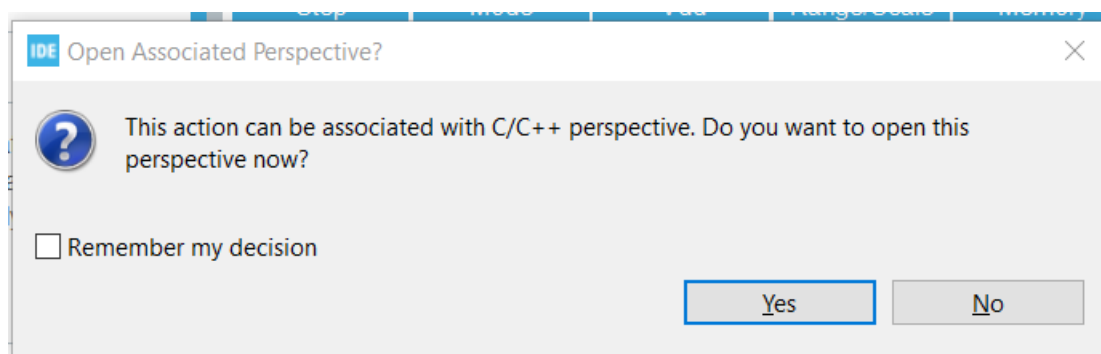
4. Assign a user label to the pin (e.g., LED_PIN, BUTTON_PIN) for easy identification in code
5. Select High Speed Clock HSE



6. Settings for Debug



7. Click the "Generate Code" or **ALT + K** button to create initialization code



HAL Library GPIO Functions

The STM32 HAL (Hardware Abstraction Layer) library provides high-level functions to control GPIO pins easily. The most commonly used GPIO functions are:

Function	Description
HAL_GPIO_WritePin(GPIOx, GPIO_Pin, PinState)	Sets or clears the selected data port bit
HAL_GPIO_ReadPin(GPIOx, GPIO_Pin)	Reads the specified input port pin
HAL_GPIO_TogglePin(GPIOx, GPIO_Pin)	Toggles the specified GPIO pin
HAL_Delay(ms)	Provides accurate delay in milliseconds

Note: GPIOx can be GPIOA, GPIOB, GPIOC, etc. GPIO_Pin can be GPIO_PIN_0 to GPIO_PIN_15. PinState can be GPIO_PIN_SET or GPIO_PIN_RESET.

Example: Writing to a GPIO Pin

```
// Turn LED ON (connected to PB11)
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, GPIO_PIN_SET);

// Turn LED OFF
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, GPIO_PIN_RESET);

// Toggle LED
HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_11);
```

Example: Reading from a GPIO Pin

```
// Read button state (connected to PC6)
GPIO_PinState buttonState;
buttonState = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_6);

if(buttonState == GPIO_PIN_SET)
{
    // Button is pressed (assuming pull-down configuration)
}
else
{
    // Button is not pressed
}
```

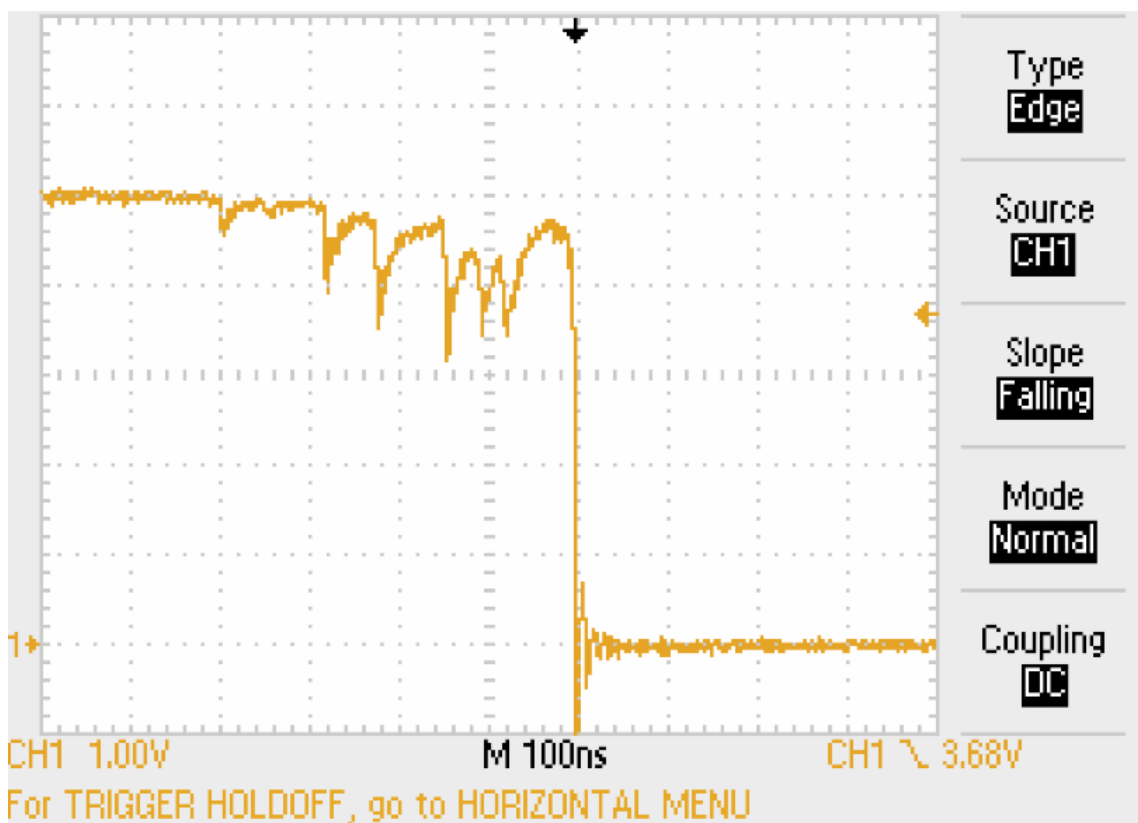
Button Debouncing

When a mechanical push button is pressed or released, the contacts do not make or break cleanly. Instead, they bounce against each other multiple times before settling into a stable state. This bouncing can last from a few microseconds to several milliseconds and can cause the microcontroller to register multiple button presses from a single physical press.

Why Debouncing is Important

Without debouncing, a single button press may be interpreted as multiple presses, leading to:

- Incorrect counting in counter applications
- Multiple unwanted state changes
- Erratic system behavior
- Poor user experience



A robust approach uses state tracking to ensure the button remains in the new state for a minimum duration.

```
uint32_t button_press_time = 0;
uint8_t button_pressed = 0;
#define DEBOUNCE_TIME 50 // 50ms

while(1)
{
    if(HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_6) == GPIO_PIN_SET)
    {
        if(!button_pressed)
        {
            if(button_press_time == 0)
            {
                button_press_time = HAL_GetTick();
            }
            else if((HAL_GetTick() - button_press_time) >= DEBOUNCE_TIME)
            {
                // Button press confirmed
                button_pressed = 1;
                // Execute your action here
                HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_11);
            }
        }
    }
    else
    {
        button_press_time = 0;
        button_pressed = 0;
    }
}
```

In-Lab Exercise

Task 1: LED Blinking

Create a C program that blinks the LED repeatedly with some delay, whereas the LED is attached at I/O pin PB11 of STM32F4 training board using HAL library functions..

Steps:

1. Create a new STM32 project in STM32CubeIDE
2. In STM32CubeMX, configure PB11 as GPIO_Output
3. Set user label as "LED_PIN"
4. Generate code
5. Write the program in main.c

Task 2: LED activation with push button

Write a C program that continuously monitors push button. When button is activated, LED will turn on otherwise remain off, whereas the LED and switch are attached at I/O pin PB11 and PC6 respectively of STM32F4 training board.

Steps:

1. Configure PB11 as GPIO_Output (LED)
2. Configure PC6 as GPIO_Input (Button)
3. Set appropriate pull-up or pull-down configuration for button
4. Generate code
5. Write the program with button reading logic

Task 3: Multiple LEDs with Two Push Buttons

Write a program that will continuously monitor two push buttons and generate the output as per the following conditions:

- a) Initially all the four LEDs are OFF.
- b) When push button 1 is pressed, the number of ON LEDs will increase by 1
- c) When push button 2 is pressed, the number of ON LEDs will decrease by 1

Steps:

1. Configure eight GPIO pins as outputs for LEDs (e.g., PB8 to PB15)
2. Configure two GPIO pins as inputs for buttons (e.g., PC6, PC7)
3. Implement button debouncing using the method discussed earlier
4. Update LED displays

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	

Instructor Signature: _____

Date: _____